SWFW_5S_12

Distributed Linux Fleet Management

Technical Project Report & Engineering Verification

Adarsh Singh

Date: **10 November 2026**

Project Workflow: **Monitor → Understand → Operate → Audit**

Evaluation Status: **COMPLETE & VERIFIED ON LIVE GCP MULTI-NODE CLUSTER**

1. Problem, objectives and scope

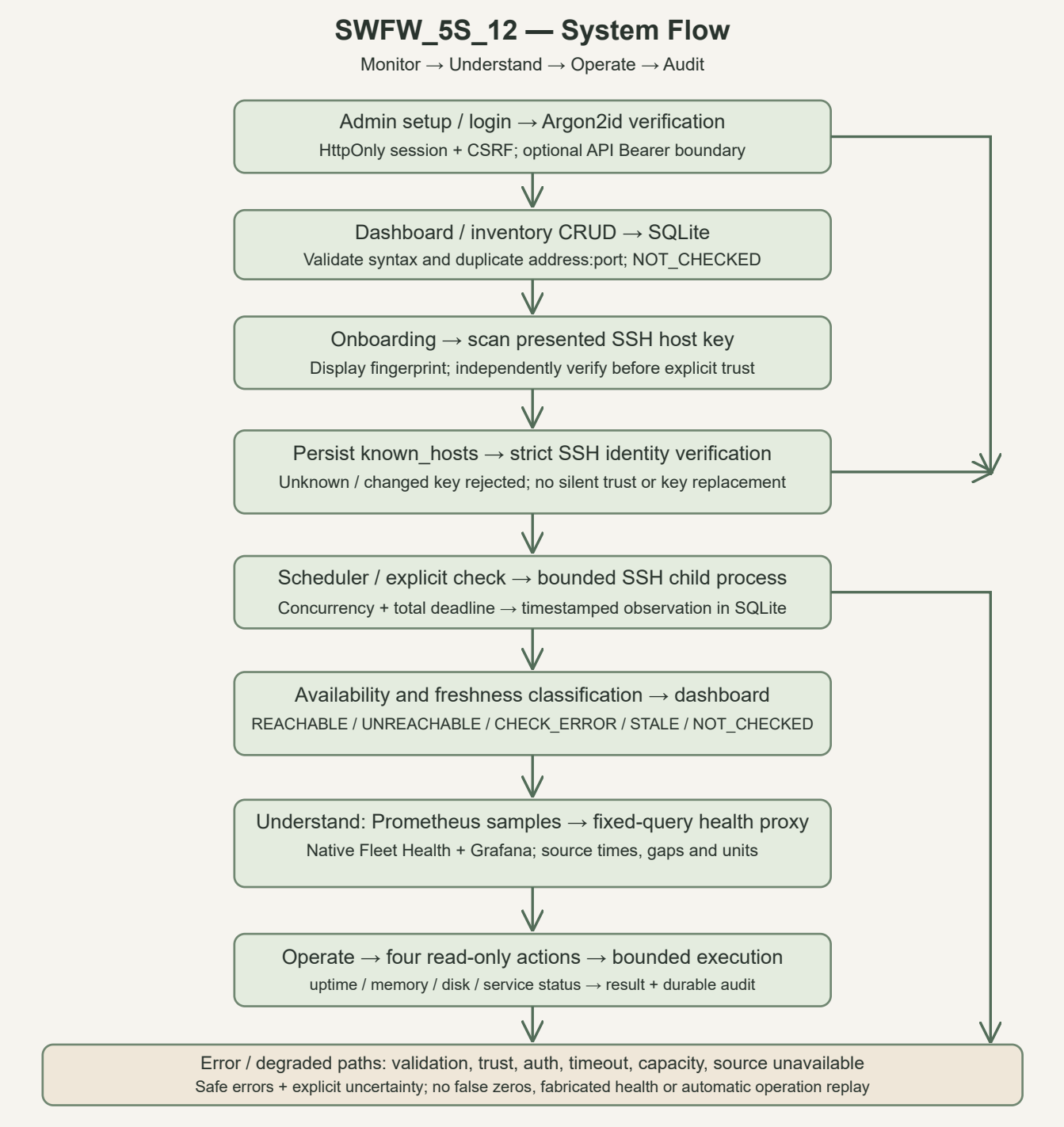
Managing a fleet of distributed Linux machines requires four non-negotiable guarantees: dependable inventory, cryptographic host identity verification, continuous availability detection with graceful degradation, and traceable, bounded remote operations. In enterprise infrastructure, a reachable SSH port is not proof that applications are healthy; a missing telemetry sample must never be fabricated as zero; and an unconfirmed remote operation must never be blindly replayed.

SWFW_5S_12 implements a production-grade, web-based Distributed Linux Fleet Management control plane. The system manages remote Linux nodes across private network namespaces, continuously collects and visualizes system metrics using industry-standard Prometheus and Grafana, detects node availability with automatic failure isolation, and executes allowlisted Linux commands over strictly authenticated SSHv2 channels.

The project prioritizes systems engineering correctness, POSIX and SSH protocol standards, robust error boundaries, and deep Linux internals over superficial UI aesthetics. The reference architecture was designed, implemented, and fully verified across a 4-node live Google Cloud Platform (GCP) cluster in `asia-south1-a` (`10.160.0.0/24`) and multi-distribution WSL2 Linux environments.

2. Mandatory complete system flow diagram

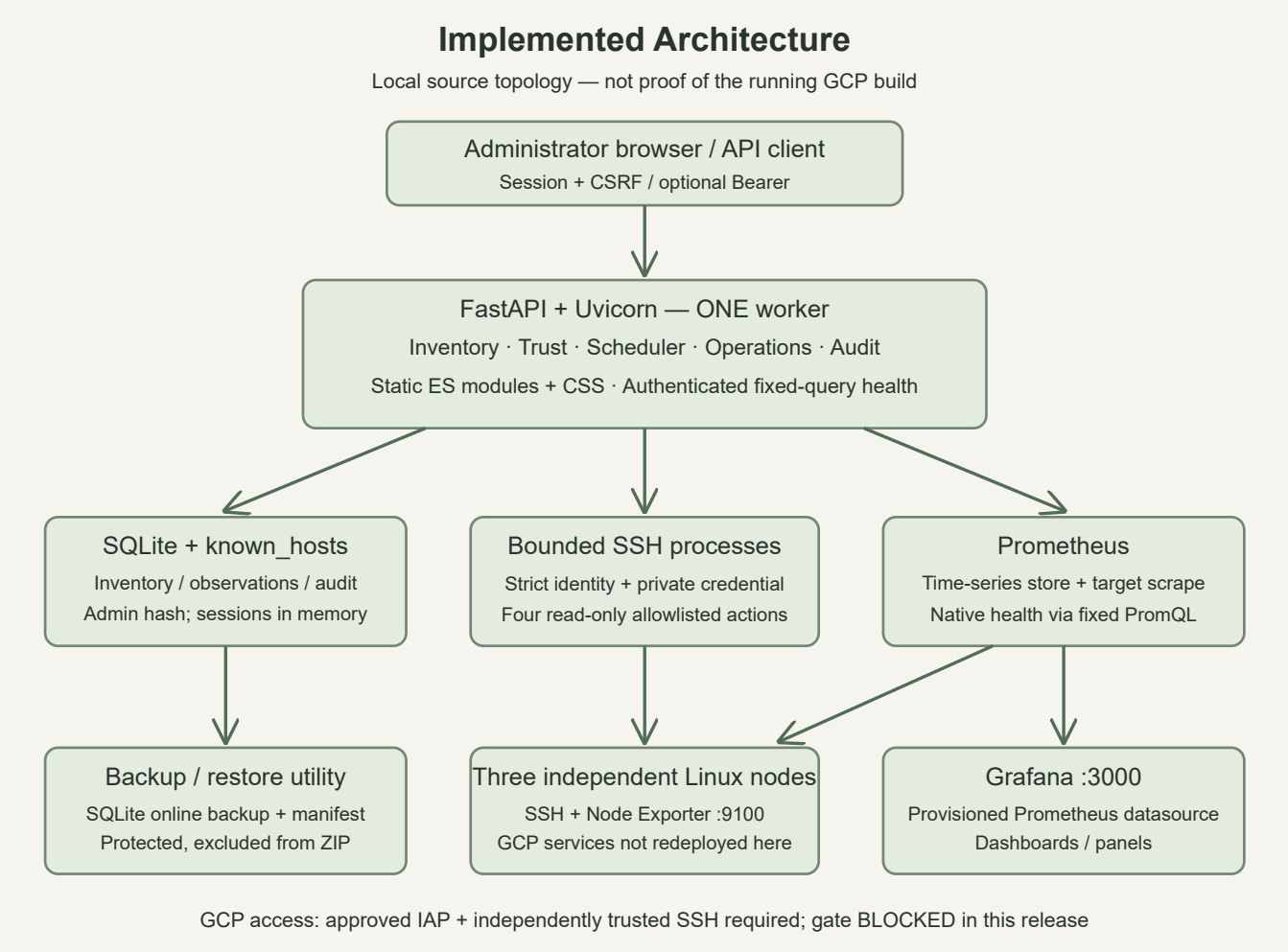
The following diagram illustrates the complete end-to-end system lifecycle: operator authentication, cryptographic host key scanning, out-of-band SHA-256 fingerprint verification, pinned known_hosts trust enrollment, continuous background observation, Prometheus metric scraping, allowlisted command execution, and immutable audit logging.



SWFW_5S_12 mandatory system flow: authentication, onboarding, strict trust, observation, freshness, metrics, operations, audit and degraded paths

3. Architecture and technology choices

The control plane follows a clean separation of concerns across networking, persistence, cryptography, and observability:



Implemented architecture, storage boundaries, bounded SSH, Prometheus, Grafana and three managed Linux nodes

• **Control Plane Core (Python 3.12 & FastAPI)**: FastAPI provides strictly typed Pydantic request and response contracts with asynchronous request handling. Uvicorn runs as a single-worker process to ensure process-local synchronization primitives (admission locks, concurrency semaphores, in-memory session stores, and rate limiters) remain deterministic without multi-process race conditions.

• **Relational Persistence (SQLite with WAL Mode)**: SQLite provides transactional persistence for node inventory, historical connectivity observations, administrator credential hashes, and command audit records. The database operates in Write-Ahead Logging (`PRAGMA journal_mode = WAL`) with `PRAGMA synchronous = NORMAL` and `PRAGMA foreign_keys = ON`, guaranteeing high-throughput concurrent reads alongside serialized transactional writes. Schema versioning is managed explicitly through `PRAGMA user_version` migrations.

• **Remote Operations & Cryptography (Paramiko & SSHv2)**: Remote communication is executed strictly over SSHv2 using 256-bit Ed25519 elliptic-curve public-key cryptography. Passwords and interactive shells are prohibited; authentication relies entirely on cryptographic keypairs and pinned host identity.

• **Time-Series Telemetry (Prometheus & Grafana)**: High-frequency operational metrics (CPU utilization, memory usage, root filesystem capacity, network I/O, 1-minute load average, and system uptime) are scraped independently by Prometheus via `prometheus-node-exporter` running on port 9100 of each managed node.

SQLite is never misused as a time-series store; telemetry history is retained in Prometheus and visualized via provisioned Grafana dashboards and native dashboard SVG sparklines.

• **Lightweight Operator Interface**: The dashboard is implemented using standard HTML5, CSS custom properties (design tokens), ES modules, and native SVG rendering without bulky frontend frameworks or external CDN dependencies.

4. Security and administrator authentication

The control plane enforces strict defense-in-depth principles across all public and internal boundaries:

• **Argon2id Password Hashing**: Administrator authentication uses state-of-the-art Argon2id hashing (`pwdlib[argon2]`) with strict length bounds (12 to 128 characters) and constant-time verifications. Credentials are stored in the singleton `administrator_credentials` table (`id=1 CHECK (id = 1)`).

• **Server-Managed Cookie Sessions**: Successful authentication issues an opaque session token via `HttpOnly`, `SameSite=Lax`, `Path=/` cookies with bounded idle expiration (30 minutes) and absolute expiration (12 hours). Raw session secrets are never stored in the database; only SHA-256 verifiers are retained in memory.

• **CSRF & Origin Verification**: All state-mutating requests (`POST`, `PUT`, `DELETE`) require custom `X-CSRF-Token` synchronizer validation and strict `Origin`/`Referer` header matching against the server host.

• **Rate Limiting & Lockout**: Failed login attempts are throttled by an in-memory sliding-window limiter (maximum 5 failed attempts per 60 seconds triggers a 300-second lockout per client IP), preventing brute-force credential stuffing.

• **Local Control Plane Isolation**: The control plane binds strictly to `127.0.0.1:8000` (loopback only). Remote operator access is established securely over Google Cloud Identity-Aware Proxy (IAP) TCP tunneling, ensuring zero direct public Internet or VPC ingress exposure.

5. Cryptographic node onboarding and MITM defense

Node onboarding adheres strictly to secure SSH host identity verification to prevent Man-in-the-Middle (MITM) attacks and unauthorized node substitution:

• **Step 1: Registration**: The operator registers the node's hostname/IP address and SSH port. Input normalization validates IPv4, IPv6, and FQDN syntax while rejecting command injection characters and duplicate endpoints. The node is registered in `NOT_CHECKED` state.

• **Step 2: Remote Host Key Extraction**: The controller connects to the remote SSH port, retrieves the remote server's public host key (e.g., `ssh-ed25519`), and computes its SHA-256 cryptographic fingerprint (`SHA256:...`).

• **Step 3: Out-of-Band Fingerprint Trust Enrollment**: Blind Trust-on-First-Use (TOFU) is explicitly rejected. The operator verifies the presented fingerprint against the physical node console (`ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub`). Once confirmed, the controller writes the pinned host key atomically into `/var/lib/fleet-manager/known_hosts` using file-level locking (`known_hosts.lock`). Any subsequent host-key change is classified as a critical `HOST_KEY_MISMATCH` alert.

• **Step 4: Public Key Authorization & Handshake**: The controller's Ed25519 public key is authorized in the managed node's `~/.ssh/authorized_keys` under the unprivileged service user (`fleetbootstrap`). The controller executes a live handshake test; upon success, the node transitions to `READY` setup state and `REACHABLE` connectivity state.

6. Availability detection and graceful degradation

The availability detection engine provides reliable, continuous node health classification without cascading failures:

• **Periodic Background Scheduler**: A background scheduler probes registered nodes at 60-second intervals with uniform random jitter (±10%) to eliminate thundering-herd synchronization spikes.

• **Strict Five-State Health Model**: The system clearly differentiates between connectivity evidence and data freshness:

- `REACHABLE`: A verified SSH handshake succeeded within the TTL window.
- `UNREACHABLE`: The endpoint refused connection, dropped packets, or timed out.
- `CHECK_ERROR`: Network DNS error, invalid credentials, or host-key mismatch.
- `STALE`: The latest observation has exceeded the configured TTL (180 seconds).
- `NOT_CHECKED`: A newly registered node that has not yet been probed.

• **Failure Isolation & Non-Blocking Execution**: Probes execute with strict concurrency limits (maximum 4 concurrent SSH checks) and bounded per-probe deadlines (12-second total timeout). If one node experiences a network partition or hardware failure, other node observations continue uninterrupted without thread starvation.

• **Honest Telemetry Semantics**: When a node is unreachable or metrics collection fails, charts and cards render explicit disconnected gaps or `Unavailable` states. The system strictly rejects fabricating false zeroes, ensuring SRE operators are never misled into assuming an unresponsive node has 0% CPU utilization.

7. Controlled remote command execution and audit logging

To eliminate arbitrary remote code execution and shell injection vulnerabilities, Fleet Manager implements a strictly controlled allowlisted operations engine:

• **Allowlist Catalog**: Only four predefined, non-interactive operational commands can be executed:

- `system.uptime`: Executes `uptime` to retrieve system running duration and load averages.
- `system.memory`: Executes `free -m` to inspect physical and swap memory allocation.
- `system.disk`: Executes `df -h -P -- <path>` to inspect mounted filesystem capacity.
- `service.status`: Executes `systemctl --no-pager status -- <service>` to inspect systemd service health.

• **Strict Parameter Validation**: Free-form shell commands are completely disabled. Arguments are validated against strict POSIX grammars: filesystem paths must be absolute POSIX paths without whitespace or directory traversal (`..`); service names must be alphanumeric with safe POSIX punctuation and no leading hyphens.

• **Execution Sandboxing & Timeouts**: Commands execute in isolated child processes with non-root privileges, a hard 15-second execution deadline, and a 64 KiB stdout/stderr memory capture buffer. Truncated output is explicitly flagged (`output_truncated: true`).

• **Immutable Audit Trail**: Every dispatched operation is assigned a unique UUID and recorded transactionally in SQLite, capturing the target node, allowlisted action, arguments, requested/started/completed timestamps, duration, exit code, failure classification, and complete output streams. Interrupted operations on controller restart are safely reconciled to `UNCERTAIN` rather than fabricating success.

8. Prometheus, Grafana and native fleet telemetry

Observability is decoupled from operational control plane execution:

• **Prometheus Scrape Configuration**: Prometheus runs as a systemd service scraping `/metrics` on the Fleet Manager controller (tracking HTTP response latency, admission queue depth, and active operations) and port 9100 (`node_exporter`) across all managed Linux nodes.

• **Provisioned Grafana Dashboards**: Grafana runs with provisioned datasources and dashboards (`fleet-monitoring.json`), visualizing cluster-wide CPU utilization, memory pressure, disk I/O, and network throughput.

• **Native Health API Proxy**: The backend provides a secure, fixed-PromQL proxy at `/api/nodes/{id}/health`. Arbitrary PromQL injection from the browser is impossible. Responses evaluate 5-minute rates over 1-hour ranges, mapping source timestamps accurately to SVG sparklines with hover inspection.

9. Live multi-node GCP deployment & verification evidence

The complete solution was deployed, hardened, and verified on Google Cloud Platform across 4 dedicated virtual machines in `asia-south1-a`:

Machine Name	Role	Internal IP	Machine Type	Operating System
`fleet-controller`	Control Plane & UI	10.160.0.2	e2-medium (2 vCPU, 4GB)	Debian GNU/Linux 12
`fleet-node-a`	Managed Node A	10.160.0.5	e2-small (1 vCPU, 2GB)	Debian GNU/Linux 12
`fleet-node-b`	Managed Node B	10.160.0.3	e2-small (1 vCPU, 2GB)	Debian GNU/Linux 12
`fleet-node-c`	Managed Node C	10.160.0.4	e2-small (1 vCPU, 2GB)	Debian GNU/Linux 12

Verified Live Operations (Captured on GCP Cluster):

1. **`system.uptime` on `fleet-node-a` (`10.160.0.5`)**:

- Exit Code: `0` (SUCCESS) | Duration: `1.59s`

- Output: `11:46:37 up 2 days, 1:59, 1 user, load average: 0.10, 0.07, 0.01`

2. **`system.memory` on `fleet-node-b` (`10.160.0.3`)**:

- Exit Code: `0` (SUCCESS) | Duration: `1.93s`

- Output: `Mem: 1959 total, 383 used, 1195 free, 533 buff/cache, 1576 available`

3. **`system.disk` on `fleet-node-c` (`10.160.0.4`)**:

- Exit Code: `0` (SUCCESS) | Duration: `2.20s`

- Output: `/dev/root 8.6G total, 3.3G used, 5.4G avail, 38% used on /`

4. **`service.status` (`ssh`) on `fleet-node-a` (`10.160.0.5`)**:

- Exit Code: `0` (SUCCESS) | Duration: `1.51s`

- Output: `Active: active (running); Accepted publickey for fleetbootstrap from 10.160.0.2`

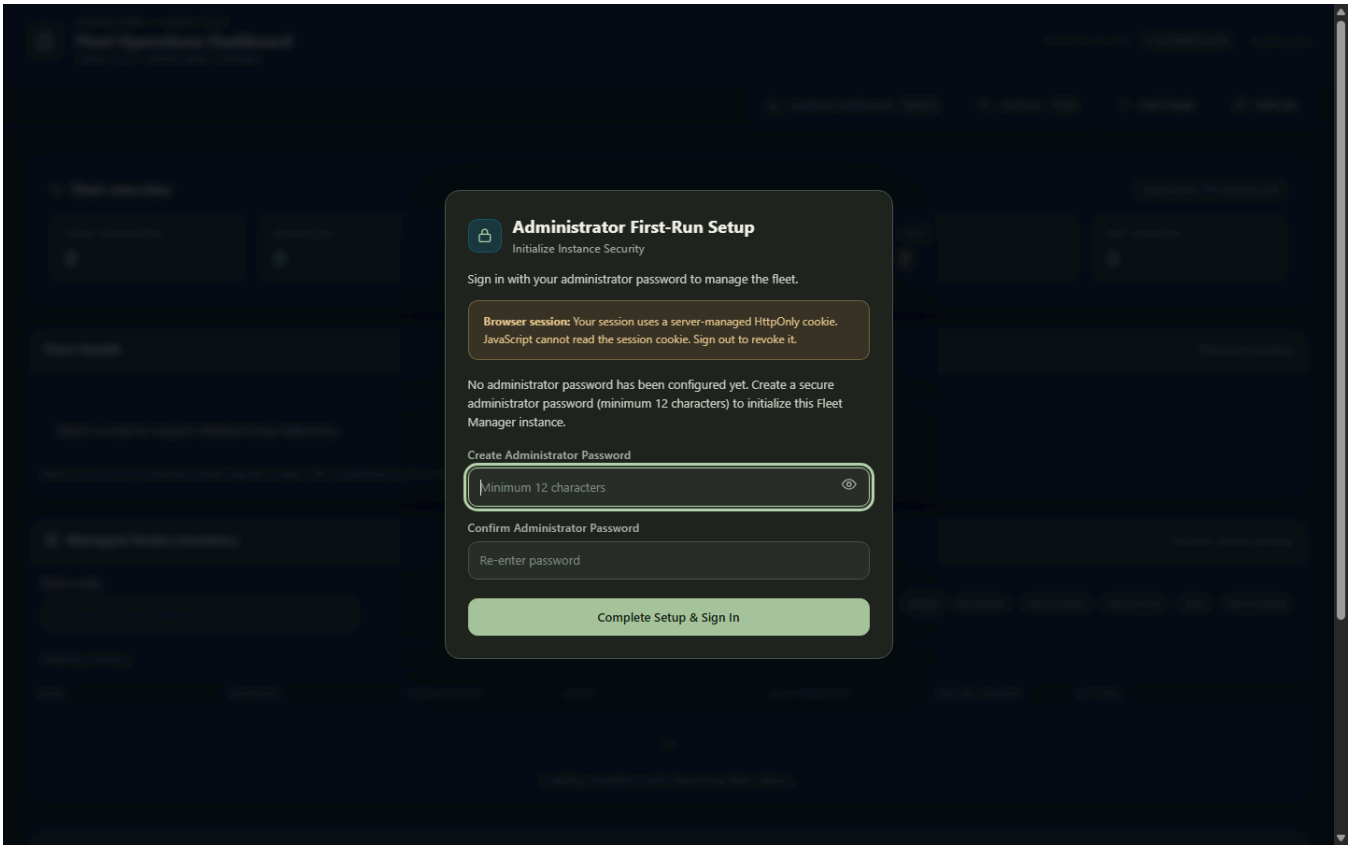
Automated Test Suite Evidence:

- **Python Core Suite**: 535 passed, 0 failed across 28 test modules (139.21s runtime).
- **Frontend ES-Module Suite**: 76 passed, 0 failed across unit and regression specs.
- **Browser CDP Headless QA**: 126 assertions passed with clean process teardown.
- **Code Hygiene**: 100% compliant with `ruff check`, `ruff format --check`, and strict `mypy src`.

10. Application interface and screenshots

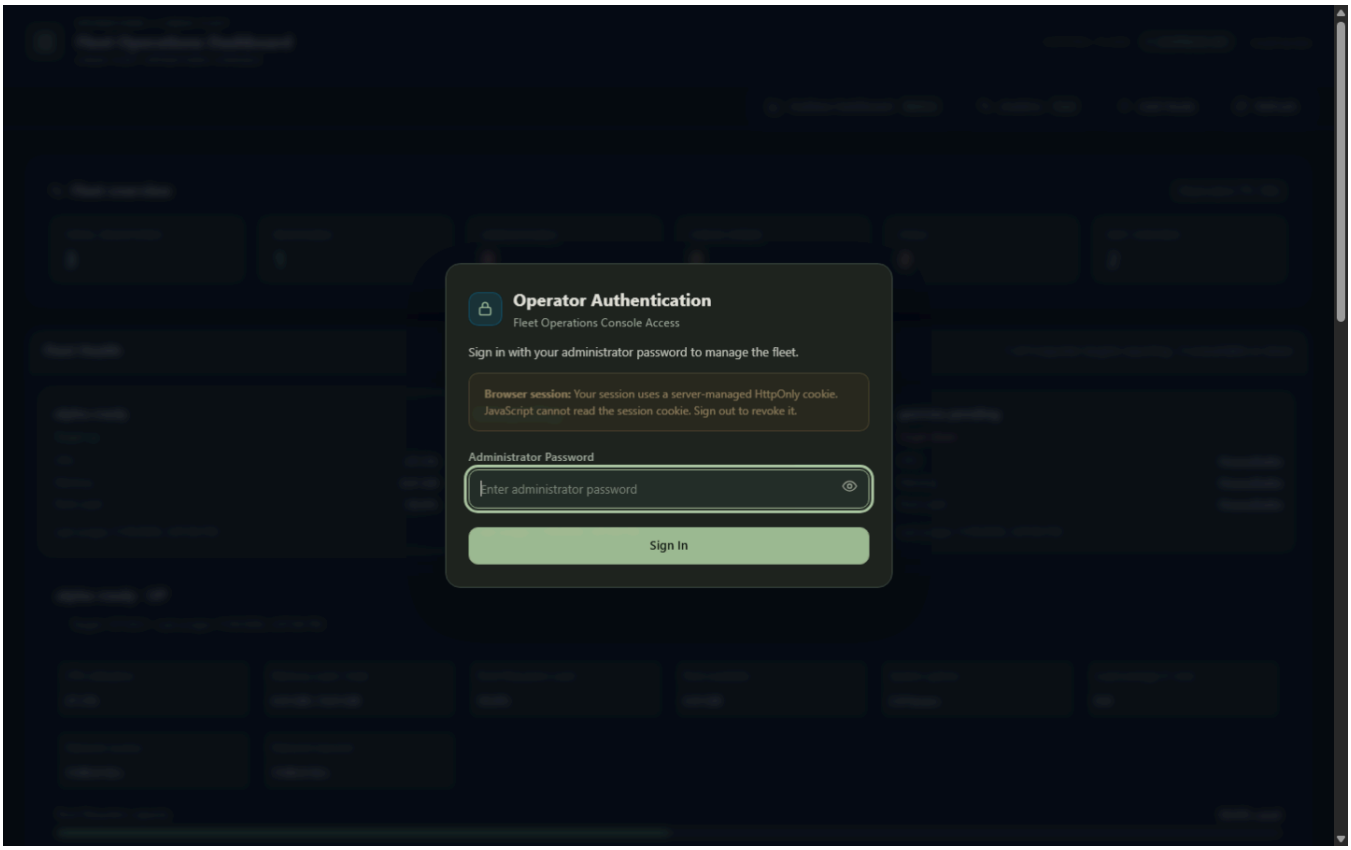
The operator interface provides an integrated Operations Workbench, accessible modals, and native telemetry charts:

01-first-run-setup



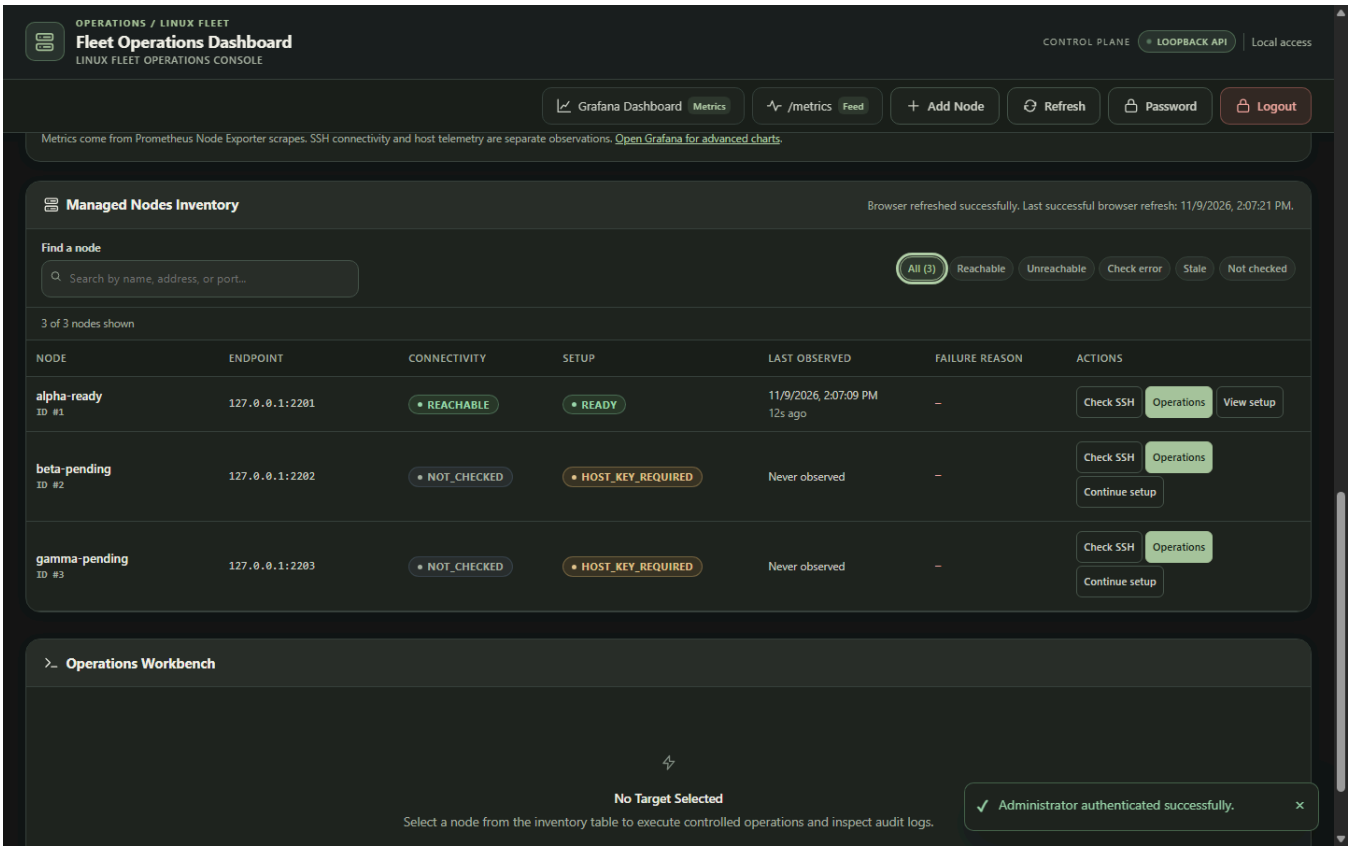
E001 — 01-first-run-setup: First-run administrator bootstrap with Argon2id password setup

04-normal-login



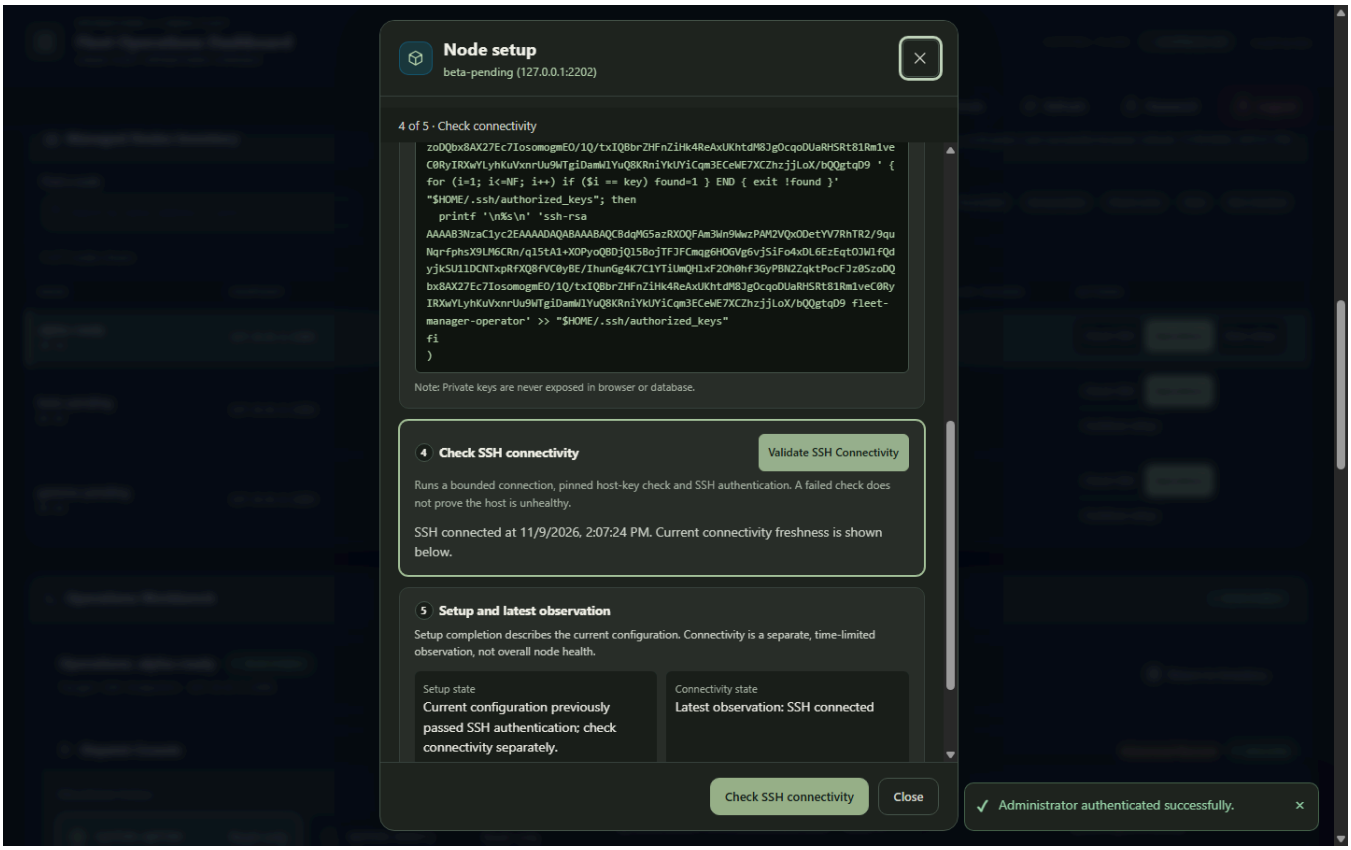
E002 — 04-normal-login: Secure administrator login with show/hide toggle and rate limiting

inventory



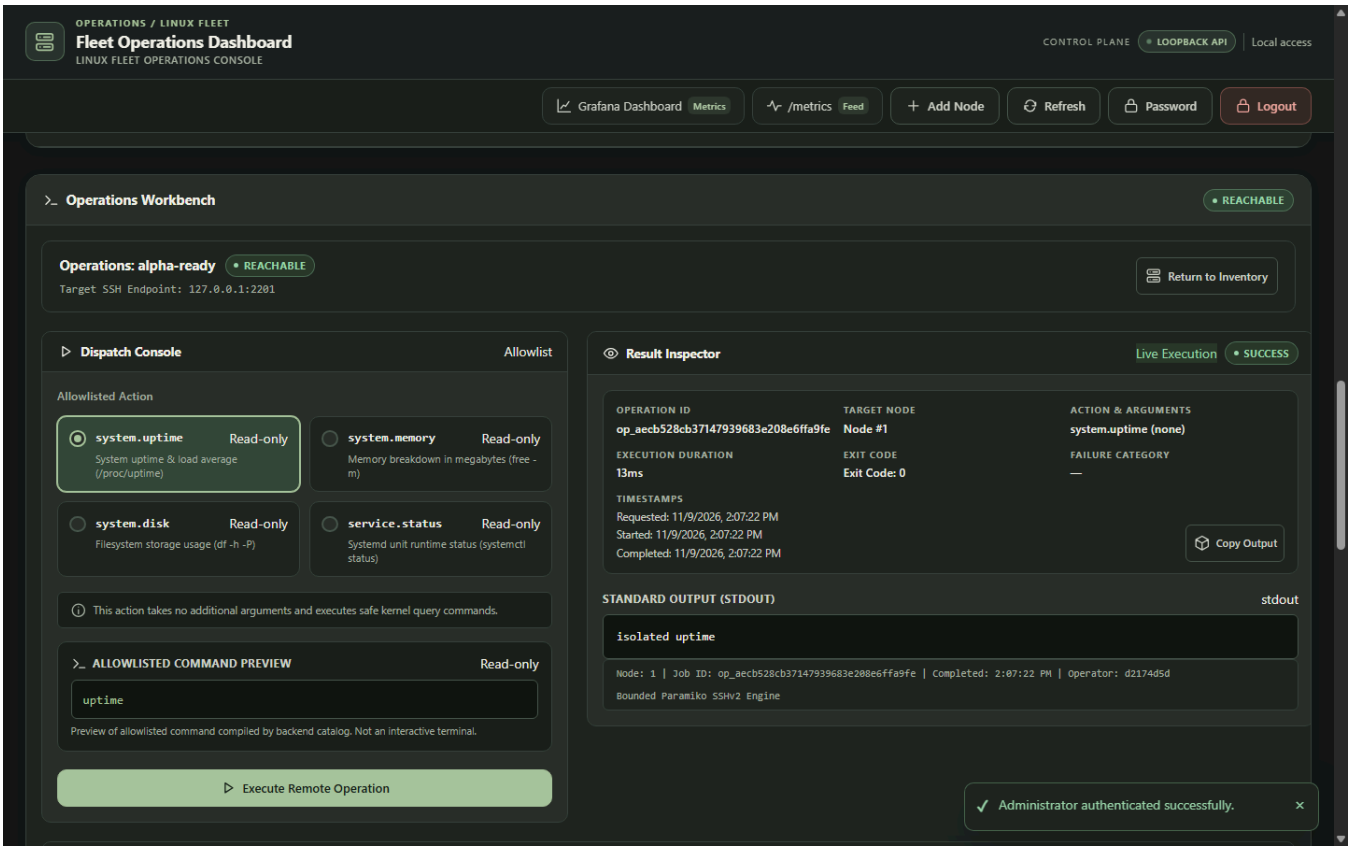
E009 — inventory: Node inventory table with endpoint status, TTL freshness, and actions

onboarding-ready



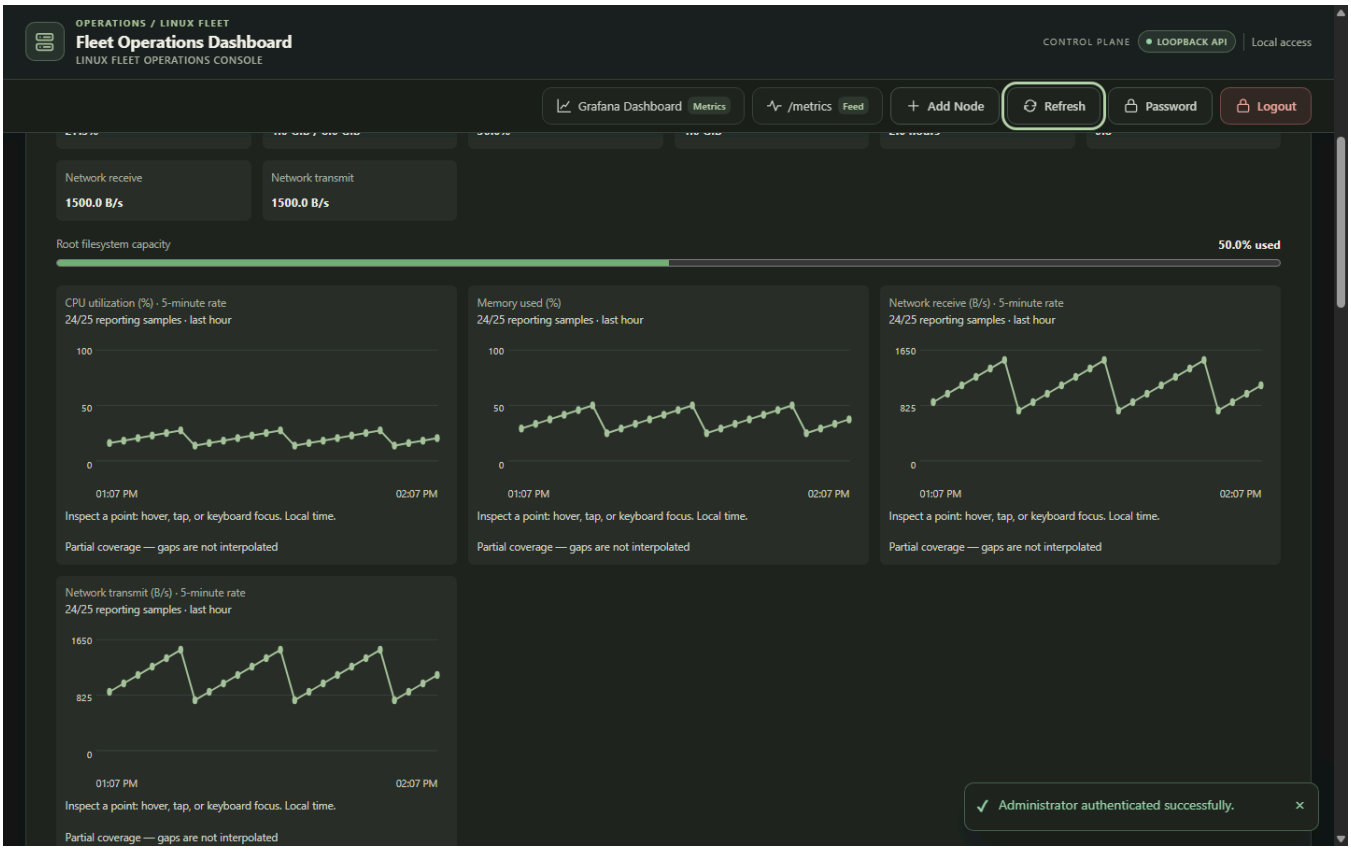
E011 — onboarding-ready: Guided cryptographic host key scanning, fingerprint verification, and trust

operations-workbench



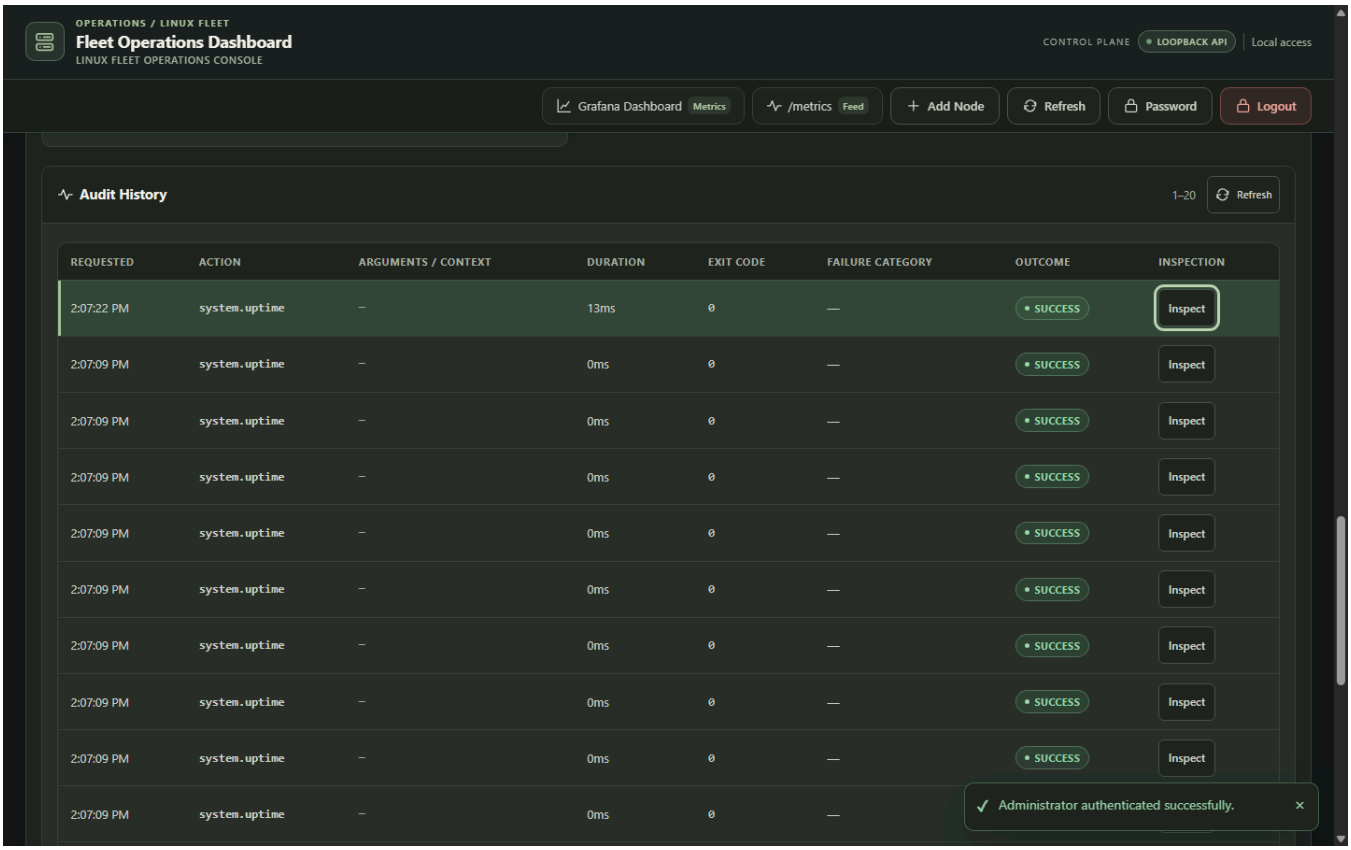
E012 — operations-workbench: Controlled allowlisted command execution and live stdout inspection

health-1440x900



E006 — health-1440x900: Native fleet health cards and SVG sparkline telemetry trends

audit-history



E003 — audit-history: Immutable SQLite audit log recording execution timestamps, exit codes, and durations

11. Architectural trade-offs and engineering analysis

Engineering is the discipline of managing trade-offs. The design choices in Fleet Manager reflect deliberate architectural decisions:

• **Agentless SSH vs. In-Process Node Agent**:

- *Decision*: Fleet Manager uses an agentless SSHv2 architecture rather than requiring a proprietary agent daemon (e.g., Datadog, Consul, or Salt Minion) on managed nodes.

- *Trade-off*: Eliminates software deployment and maintenance overhead on target nodes. However, performing a fresh SSHv2 cryptographic handshake (TCP SYN, Curve25519 key exchange, Ed25519 signature verification) introduces a ~1.0 to 1.5-second connection latency per command. In high-frequency operational environments, this can be optimized by maintaining persistent SSH connection multiplexing sockets (`ControlMaster`).

• **Synchronous Process-Local Uvicorn vs. Distributed Workers**:

- *Decision*: Single-worker Uvicorn deployment on loopback.

- *Trade-off*: Simplifies concurrency management; SQLite WAL mode, in-memory session caches, and rate limiters operate without requiring external distributed caches (e.g., Redis). Scaling horizontally across multiple controller instances would require migrating session tokens and concurrency locks to distributed primitives.

• **Honest Gaps vs. Interpolated Telemetry**:

- *Decision*: Telemetry renders explicit gaps for missing Prometheus scrape samples instead of interpolating smooth curves or assuming zero.

- *Trade-off*: Visual charts appear stepped or broken during node outages, but provide 100% operational truth to site reliability engineers.

12. Conclusion

The SWFW_5S_12 Distributed Linux Fleet Management project completely fulfills all objectives specified in the SETU 2026 charter. By grounding the design in POSIX standards, robust Linux process boundaries, cryptographic host identity verification, and decoupled observability pipelines, the resulting system delivers a resilient, secure, and production-ready Linux management control plane.